Hint Bits

From PostgreSQL wiki

Languages: | **English** • 日本語

PostgreSQL's MVCC mechanism provides a lot of useful features, but there are some confusing side effects of the implementation. One revolves around hint bit processing, which can result in heavy writes to a database table even though you're just reading from it.

Hint bits are used to mark tuples as created and/or deleted by transactions that are known committed or aborted. To determine the visibility of a tuple without these bits set, you need to consult pg_clog and possibly pg_subtrans, so it is an expensive check. On the other hand, if the tuple has the bits set, then its state is known (or, at worst, it can be calculated easily from your current snapshot, without looking at pg_clog).

There are four hint bits:

- XMIN_COMMITTED -- creating transaction is known committed
- XMIN_ABORTED -- creating transaction is known aborted
- XMAX_COMMITTED -- same, for the deleting transaction
- XMAX_ABORTED -- ditto

If neither of the XMIN bits is set, then either:

- The creating transaction is still in progress, which you can check by examining the list of running transactions in shared memory;
- You are the first one to check since it ended, in which case you need to consult pg_clog to know the transaction's status, and you can update the hint bits if you find out its final state.

If the tuple has been marked deleted, then similar remarks apply to the XMAX bits.

Any examination whatsoever of a tuple --- whether by vacuum or any ordinary DML operation --- will update its hint bits to match the commit/abort status of the inserting/deleting transaction(s) as of the instant of the examination. A plain SELECT, count(*), or VACUUM on the entire table will check every tuple for visibility and set its hint bits.

Another point to note is that the hint bits are checked and set on a per tuple basis. Although a simple scan will visit all the tuples on a page and update all their hint bits at once, piecemeal access (such as fetching single tuples via index scans) might result in many writes of the same page as various hint bits get updated over time.

Commit logging

Some details here are in src/backend/access/transam/README:

- "pg_clog records the commit status for each transaction that has been assigned an XID."
- "Transactions and subtransactions are assigned permanent XIDs only when/if they first do something that requires one --- typically, insert/update/delete a tuple, though there are a few other places that need an XID assigned."

pg_clog is updated only at sub or main transaction end. When the transactionid is assigned the page of the clog that contains that transactionid is checked to see if it already exists and if not, it is initialised.

pg_clog is allocated in pages of 8kB apiece. Each transaction needs 2 bits, so on an 8 kB page there is space for 4 transactions/byte * 8k bytes = 32k transactions.

On allocation, pages are zeroed, which is the bit pattern for "transaction in progress". So when a transaction starts, it only needs to ensure that the pg_clog page that contains its status is allocated, but it need not write anything to it. In 8.3 and later, this happens not when the transaction starts, but when the Xid is assigned (i.e. when the transaction first calls a read-write command). In previous versions it happens when the first snapshot is taken, normally on the first command of any type with very few exceptions.

This means that one transaction in every 32K writing transactions *does* have to do extra work when it assigns itself an XID, namely create and zero out the next page of pg_clog. And that doesn't just slow down the transaction in question, but the next few guys that would like an XID but arrive on the scene while the zeroing-out is still in progress. This probably contributes to reported behavior that the transaction execution time is subject to unpredictable spikes.

CLOG pages don't make their way out to disk until the internal CLOG buffers are filled, at which point the least recently used buffer there is evicted to permanent storage.

Source material:

- http://archives.postgresql.org/pgsql-patches/2005-07/msg00390.php
- http://archives.postgresql.org/pgsql-performance/2008-05/msg00182.php

Retrieved from "https://wiki.postgresql.org/index.php?title=Hint_Bits&oldid=24622"
Category: Performance

- This page was last modified on 29 April 2015, at 01:24.